

CS681 Assignment 2

Discrete Event Simulation Analysis of a Web Application

Jigyasa Chouhan
Ramachandran S

Computer Science and Engineering

Introduction

- Queuing systems are fundamental to modern computing systems
- Widely used in:
 - Web servers
 - Operating systems
 - Distributed systems
- Performance depends on:
 - Request arrival patterns
 - Service mechanisms
 - Scheduling policies
- **Goal:** Build a flexible simulator to model and analyze such systems

Users → Request → Server → Response

Motivation & Problem Statement

- Real-world systems handle large volumes of concurrent requests
- Performance is affected by:
 - System load (number of users)
 - Scheduling policies
 - Resource constraints (threads, cores, buffers)
- Analytical models (e.g., Little's Law, MVA) provide insights
 - Often rely on simplifying assumptions
 - Difficult to model complex, realistic systems
- **Problem:**
 - How do we evaluate performance under realistic and configurable settings?
 - How well do theoretical models match real system behavior?
- **Solution:**
 - Design a configurable, event-driven queuing network simulator
 - Empirically validate theoretical results

System Overview

- We model a **closed queuing network**
 - Fixed number of users circulating in the system
 - Each user generates requests repeatedly
- The system consists of:
 - **Client Nodes:** Generate requests and model user think time
 - **Server Nodes:** Process requests using threads and CPU cores
- Requests flow through the network:
 - Routing determined by a probabilistic **adjacency matrix**
 - Supports multiple servers and arbitrary topologies
- **Key Idea:**
 - Simulate realistic system behavior using configurable components

Clients → Servers → Clients (Closed Loop)

Architecture of the Queuing Network

- The system is composed of interconnected **Client** and **Server** nodes
- Each server models a multi-stage processing pipeline:
 - **Receiver Queue:** Buffers incoming requests
 - **Thread Pool:** Assigns requests to worker threads
 - **Worker:** Contains multiple CPU cores
 - **Core Buffers:** Queue threads waiting for execution
- Supports realistic features:
 - Limited buffer sizes (can cause drops)
 - Multiple scheduling policies (FCFS, SJF, RR)
 - Context switching in cores

Clients → Receiver Queue → Threads → Core Buffers → Cores



Next Node (via probabilistic routing)

Event-Driven Simulation Model

- The simulator is based on a **discrete-event model**
- System evolves through a sequence of events:
 - ARRIVAL
 - THREAD ARRIVAL
 - THREAD PROCESS
 - CONTEXT SWITCH
 - DEPARTURE
 - TIMEOUT
- Events are managed using a **priority queue**
 - Ordered by timestamp
 - Earliest event processed first

Request Lifecycle in the System

- Each user repeatedly generates requests
- Lifecycle of a request:

Client → Arrival → Thread Assignment → Core Assigned → Departure →
Next Node / Think Time

- Requests may:
 - Complete successfully
 - Timeout and retry
 - Get dropped due to full buffers
- Closed-loop behavior:
 - After completion, user waits (think time) and generates next request

Key Components: Clients, Servers, Requests

- **Clients:**

- Generate requests
- Model think time between requests
- Handle timeouts and retries

- **Requests:**

- Represent individual jobs in the system
- Track:
 - Arrival time, Departure time
 - Remaining service time
 - Timeout / dropped status

- **Servers:**

- Process requests using threads and CPU cores
- Maintain queues and buffers
- Implement scheduling policies

Server Architecture: Threads, Cores, Buffers

- Each server models a multi-stage processing system:
 - **Receiver Queue:**
 - Stores incoming requests
 - Limited capacity (can cause drops)
 - **Thread Pool:**
 - Assigns requests to threads
 - **Worker (CPU Cores):**
 - Multiple cores process threads
 - Each core has its own buffer
 - **Buffers:**
 - Thread queue (global)
 - Core-level queues
- Supports context switching and scheduling policies

Server Architecture

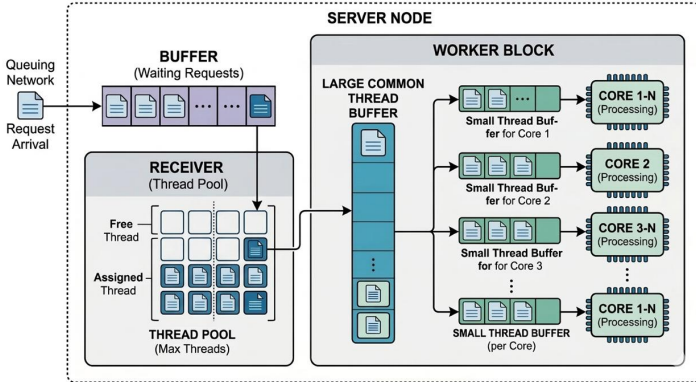


Figure: Server Node

Scheduling Policies

- The simulator supports multiple scheduling policies at server cores:
 - **First-Come First-Serve (FCFS)**
 - Requests processed in order of arrival
 - No preemption or context switching
 - **Shortest Job First (SJF)**
 - Requests with smaller service time are prioritized
 - Reduces average response time
 - **Round Robin (RR)**
 - Time-sliced execution with context switching
 - Fairness across requests

Distributions Supported

- The simulator supports multiple probability distributions:
 - **Uniform Distribution**
 - Values sampled within a fixed range
 - **Exponential Distribution**
 - Models memoryless inter-arrival/service times
 - **Deterministic Distribution**
 - Constant value for all samples
 - **Normal Distribution**
 - Models variability around a mean
 - **Multinomial Distribution**
 - Used for probabilistic routing between nodes
- Applied to:
 - Think time
 - Service time
 - Timeouts
 - Network routing

Configurable Simulation Design (via JSON)

- The simulator is fully configurable using a **JSON input file**
- Key configurable aspects:
 - **Simulation parameters:**
 - Number of runs, simulation time
 - **Client configuration:**
 - Number of users (range + step size)
 - Think time distribution
 - Timeout behavior
 - **Server configuration:**
 - Threads, cores, buffer sizes
 - Scheduling policy
 - Service time distribution
 - **Network structure:**
 - Defined via an adjacency matrix
 - Enables arbitrary routing between nodes
- **Key Advantage:**
 - Easily modify system behavior without changing code

Running the Simulation

- The Makefile provided with the code appropriately makes the object files to run the simulation
- Usage (After running make):

```
./networksim configFile
```

- After running this, the simulation runs for number of users varied from the min to max provided with the step size from the configFile. It prints all the stats into the specified output csv file in the format:
user_count, avg_response_time, good_throughput, bad_throughput, total_throughput, drop_percentage, jserver_i_utilization, ci_90_lower, ci_90_upper, ci_95_lower, ci_95_upper, ci_99_lower, ci_99_upper

Running the Simulation

Into the Code

- The run method in NetworkSim first of all pushes request arrival events into the event priority queue at timestamp 0
- The event queue is popped and each event is processed by the Node at which it occurs with respect to the provided simulation. The processing functions of client and server nodes take care of how exactly to process each event.
- With the processing of each event, more events are added to the event queue until the time stamp hits the max time specified in the sim object
- The print stats function accumulates relevant data and the actual statistical values are computed in the main function from where they're printed into the output file

Performance Metrics Collected

- **User Load**

- `user_count`: Number of active users in the system

- **Response Time**

- `avg_response_time`: Average time from request arrival to completion

- **Throughput**

- `good_throughput`: Successfully completed requests per unit time
- `bad_throughput`: Timed-out requests per unit time
- `total_throughput`: Total completed requests per unit time

- **Reliability**

- `drop_percentage`: Fraction of requests dropped due to buffer limits

- **Resource Utilization**

- `server_utilizations`: CPU utilization of server nodes
- `average_number_of_requests_in_server`: Average number in server

- **Confidence Intervals (Response Time)**

- `ci_90_lower`, `ci_90_upper`: 90% Confidence Interval
- `ci_95_lower`, `ci_95_upper`: 95% Confidence Interval
- `ci_99_lower`, `ci_99_upper`: 99% Confidence Interval

- **Little's Law Verification**

- `N_calculated_from_little_s_law` ($N = \lambda \cdot R$)
- `N_calculated_from_simulation`

- **Mean Value Analysis (MVA)**

- `R_calculated_from_mva`
- `R_calculated_from_simulation`

- **Goal**

- Validate simulation correctness using theoretical models
- Ensure statistical reliability of results

Experiment

Experiments with the Simulator

- We shall run experiments similar to the ones we ran in Assignment 1 to compare the performance of our simulation with respect to a real life system. The aim is to match the graphs/trends of the relevant values being compared
- In assignment 1, we had only considered only a system with one ServerNode, thus the following results are with a similar configuration of the simulator
- But note that the simulation is capable of simulating more complex setups involving multiple server nodes with feedback and branching involved.

Experiment Setup (Users, Runs, Parameters)

- **Simulation Configuration:**

- Simulation time: 300 s
- Number of runs: 1

- **Client Configuration:**

- Number of users: 1 to 350 (step size = 1)
- Think time: Deterministic (6 s)
- Timeout: Deterministic (5 s), no retries

- **Server Configuration:**

- Threads: 100, Cores: 1
- Large buffers (receiver + thread queues)
- Scheduling policy: FCFS
- Service time: Deterministic (0.035 s)

- **Network Topology:**

- Single client and single server
- Closed loop system (Client $\leftrightarrow$ Server)

- **Goal:**

- Replicate conditions from Assignment 1 for validation

Experiment Setup

Configuration Used

```
{
  "num_runs": 1,
  "num_nodes": 2,
  "max_time": 300,
  "nodes": [
    {
      "node_type": "CLIENT",
      "min_num_users": 5,
      "max_num_users": 350,
      "step_size": 5,
      "think_time_distribution_type": "DETERMINISTIC",
      "think_time_distribution_params": [0],
      "min_timeout": 5,
      "timeout_distribution_type": "DETERMINISTIC",
      "timeout_distribution_params": [0],
      "resend_on_timeout": false
    },
    {
      "node_type": "SERVER",
      "num_threads": 100,
      "receiver_buffer_size": 1000,
      "total_cores": 1,
      "thread_buffer_size": 1000,
      "core_buffer_size": 0,
      "scheduling_policy": "FCFS",
      "core_context_switch_time": 5,
      "core_context_switch_overhead": 0.01,
      "service_time_distribution_type": "DETERMINISTIC",
      "service_time_distribution_params": [0.035]
    }
  ],
  "adjacency_matrix": [
    [0, 1],
    [1, 0]
  ],
  "output_file": "assign1.csv"
}
```

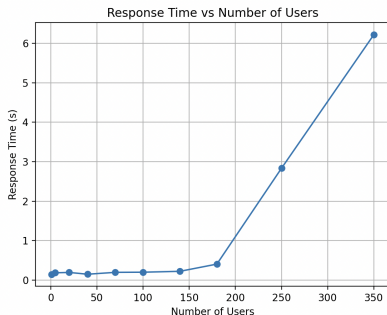
Figure: Configuration file used for experiments

Results: Assignment 1 vs Simulation

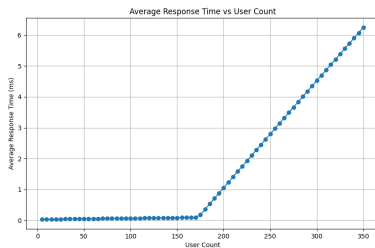
Assignment 1 vs Simulator

Response Time vs Number of Users

Assignment 1



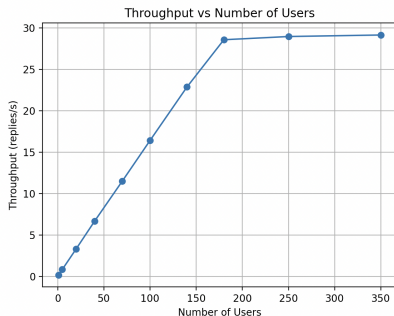
Simulator



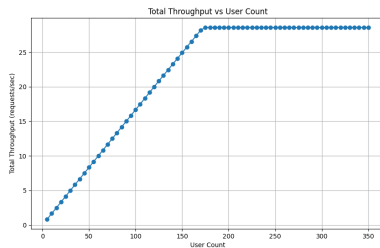
Assignment 1 vs Simulator

Throughput vs Number of Users

Assignment 1



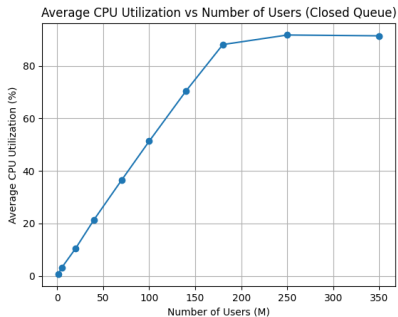
Simulator



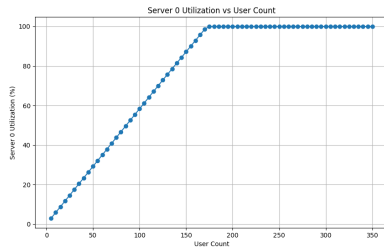
Assignment 1 vs Simulator

Server Utilization vs Number of Users

Assignment 1



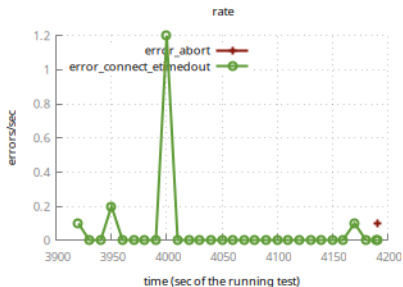
Simulator



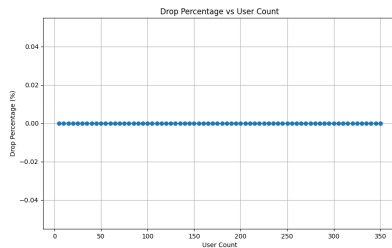
Assignment 1 vs Simulator

Drop Percentage vs Number of Users

Assignment 1



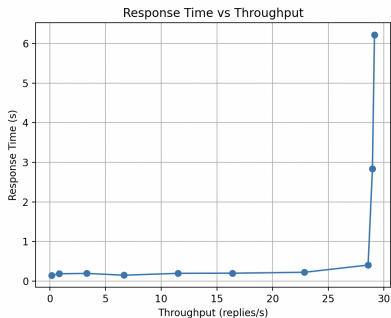
Simulator



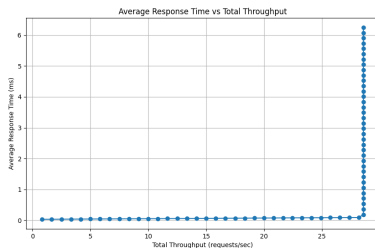
Assignment 1 vs Simulator

Response Time vs Throughput

Assignment 1



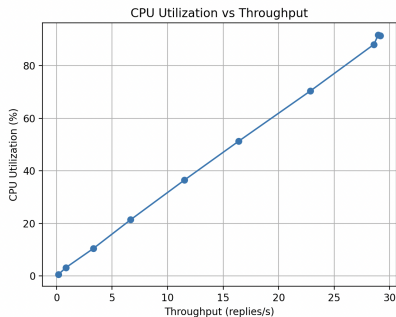
Simulator



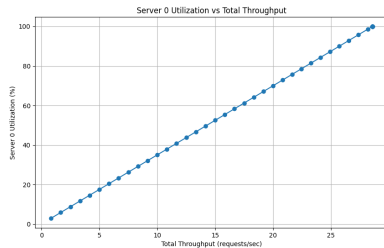
Assignment 1 vs Simulator

Throughput vs Server Utilization

Assignment 1



Simulator



Results: Theoretical Validation

Verifying Little's Law

Theory of Little's Law

- Little's Law states:

$$N = \lambda \cdot T$$

where:

- N = average number of requests in the system
 - λ = throughput (arrival rate)
 - T = average response time
- To verify this, we compute N directly from the simulation.
 - Idea:
 - Measure how long each request stays in the system
 - Aggregate this over the entire simulation
 - This gives a time-averaged estimate of the number of requests in the system.

Verifying Little's Law

Measurement in Simulation

- For each request:
 - Record arrival time at the server
 - Record departure time from the server
 - Compute:

$$\text{time_in_system} = t_{\text{departure}} - t_{\text{arrival}}$$

- Accumulate over all completed requests:

$$\sum (t_{\text{departure}} - t_{\text{arrival}})$$

- Compute average number in system:

$$N = \frac{\sum \text{time_in_system}}{\text{total simulation time}}$$

- This matches closely with $N = \lambda \cdot T$, validating Little's Law.

Verifying Little's Law

Measured values

User Count	Avg Response Time (T)	Throughput (λ)	$N_{\text{calculated}}$	N_{measured}
10	0.03815	1.66667	0.06358	0.06358
50	0.05215	8.33333	0.43458	0.43458
100	0.06965	16.6667	1.16083	1.16083
150	0.08734	24.9067	2.17548	2.17548
200	1.05874	28.3767	30.2483	30.2483

- The measured value of N matches the calculated value ($\lambda \cdot T$).
- This confirms that the simulator obeys Little's Law.

Mean Value Analysis

Theory of MVA

- Mean Value Analysis (MVA) is used to analyze closed queueing systems.
- It provides a recursive way to compute performance metrics as the number of users increases.
- In our setup, we use the following MVA relation:

$$R(N) = S \cdot (1 + Q(N - 1))$$

- Where:
 - $R(N)$ = response time at N users
 - S = average service time
 - $Q(N - 1)$ = average number of requests in the system with $N - 1$ users
- This formula captures the effect of queueing delay as load increases.

Mean Value Analysis

Implementation in Simulation

- For each user count N , we compute $R(N)$ using MVA.
- Steps followed:
 - Use the measured value of $Q(N - 1)$ from the simulation.
 - Compute average service time S :
 - Track service time of all completed requests
 - Take the average across requests
 - Apply the MVA formula:

$$R(N) = S \cdot (1 + Q(N - 1))$$

- This gives a predicted response time using analytical modeling.

Mean Value Analysis

Results

Number of Users	Service Time (S)	$Q(N - 1)$	$R_{\text{calculated}}$	R_{measured}
10	0.035	0.03675	0.0369845	0.03815
50	0.035	0.423033	0.0498062	0.05215
100	0.035	1.14345	0.0750208	0.06965
150	0.035	2.15238	0.110333	0.0873454
200	0.035	29.2516	1.05881	1.05874
300	0.035	128.347	4.52716	4.52686

- $R_{\text{calculated}} = S \cdot (1 + Q(N - 1))$
- Close agreement is observed between calculated and measured response times.

Thank You